

Comunicación RS232.

Paula Quintero y Ediem Valero.
Comunicación Digital.
Universidad Militar Nueva Granada
Bogotá, Colombia
Agosto 19, 2026

Resumen- En este laboratorio se trabajó con la comunicación serial UART utilizando tarjetas Raspberry Pi Pico y programación en MicroPython. Primero se realizaron diferentes configuraciones de UART y se enviaron varios caracteres para observar y analizar cómo se transmitía cada uno. También se realizaron mediciones del tiempo de los bits y se compararon los valores obtenidos experimentalmente con los valores teóricos para calcular el porcentaje de error.

Después, se realizó la comunicación entre dos Raspberry Pi Pico, utilizando una como transmisor y la otra como receptor. Se comprobó que los datos enviados desde una tarjeta podían ser recibidos correctamente por la otra. Además, se conectó una pantalla OLED al sistema receptor para mostrar mensajes sobre el estado de la comunicación.

Por último, se desarrolló un programa para generar archivos de texto con diferentes cantidades de líneas, entre 1 y 1000, y enviarlos por medio de la comunicación UART. Se realizaron varias pruebas cambiando la cantidad de líneas y el intervalo de tiempo entre cada envío. Con estas pruebas se midieron los tiempos de transmisión y recepción y se observó que, cuando el intervalo era muy pequeño, podían presentarse pérdidas de datos. En general, la práctica permitió comprender de una forma más clara cómo funciona la comunicación UART y algunos de los factores que pueden afectar la transferencia de información entre dos dispositivos.

I. INTRODUCCIÓN

Actualmente, la comunicación entre dispositivos electrónicos es muy importante, ya que permite enviar y recibir información entre diferentes sistemas. Uno de los métodos utilizados para realizar esta comunicación es UART, un tipo de comunicación serial que permite intercambiar datos entre dos dispositivos mediante líneas de transmisión y recepción. Para que la comunicación funcione correctamente, es necesario configurar algunos parámetros, como la velocidad de transmisión, los bits de datos, la paridad y los bits de parada.

En este laboratorio se trabajó con la comunicación UART utilizando tarjetas Raspberry Pi Pico y programación en MicroPython. El propósito de la práctica fue comprender cómo se realiza la transmisión de datos entre dispositivos y observar algunos de los factores que pueden afectar la comunicación. Para esto, primero se analizaron las señales generadas al

transmitir diferentes caracteres y se realizaron mediciones de los tiempos de los bits, comparando los resultados experimentales con los valores teóricos.

Después, se estableció una comunicación entre dos Raspberry Pi Pico, utilizando una como transmisor y la otra como receptor. Esto permitió comprobar el envío y la recepción de caracteres y mensajes entre ambos dispositivos. También se utilizó una pantalla OLED en el receptor para mostrar información relacionada con el estado de la comunicación.

Finalmente, se realizó una prueba de transferencia de archivos de texto mediante UART. Se desarrollaron programas para generar archivos con diferentes cantidades de líneas, entre 1 y 1000, enviarlos desde una Raspberry Pi Pico y guardarlos en la otra. Además, se realizaron varias pruebas cambiando la cantidad de datos y el tiempo entre cada envío para medir cuánto tardaba la transferencia y observar si se presentaban pérdidas de información.

En este informe se presenta el desarrollo de las actividades realizadas durante la práctica, los programas utilizados, las mediciones obtenidas y el análisis de los resultados. De esta manera, se busca comprender de forma práctica el funcionamiento de la comunicación UART y cómo diferentes parámetros pueden influir en la transmisión correcta de la información.

II. MARCO TEÓRICO

1. **Comunicación serial:** La comunicación serial es un método utilizado para enviar información entre dos dispositivos electrónicos. En este tipo de comunicación, los datos se transmiten bit por bit a través de una línea de comunicación. Para que la transmisión funcione correctamente, el transmisor y el receptor deben utilizar una configuración compatible. La comunicación serial es utilizada en diferentes dispositivos electrónicos, especialmente en sistemas con microcontroladores, ya que permite intercambiar información de una forma sencilla. [1]
2. **Comunicación UART:** La comunicación UART, que significa Universal Asynchronous Receiver-Transmitter, es un sistema de comunicación serial asíncrona. En este tipo de comunicación no se utiliza una señal de reloj

compartida entre el transmisor y el receptor. Por esta razón, ambos dispositivos deben configurarse con los mismos parámetros para poder interpretar correctamente la información.

Los parámetros principales de una comunicación UART son:

Baudrate: indica la velocidad de transmisión configurada para la comunicación.

Bits de datos: cantidad de bits que representan la información enviada.

Paridad: bit opcional utilizado para realizar una verificación básica de errores.

Bits de parada: indican el final de la trama transmitida. [2]

En la práctica se realizaron diferentes configuraciones de estos parámetros para observar cómo cambian las características de la transmisión.

3. **Estructura de una trama UART:** Cuando se transmite un carácter mediante UART, este se organiza en una trama. Generalmente, la transmisión comienza con un bit de inicio, seguido por los bits de datos, un posible bit de paridad y uno o varios bits de parada.

El bit de inicio permite indicar al receptor que comenzará una transmisión. Después se transmiten los bits correspondientes al dato y, finalmente, los bits de parada indican que la trama ha terminado.

La cantidad total de bits transmitidos depende de la configuración utilizada. Por esta razón, al analizar una señal UART es necesario tener en cuenta todos los parámetros configurados en el programa. [3]

4. **Tiempo de bit y velocidad de transmisión:** El tiempo que dura cada bit está relacionado con la velocidad de transmisión. De forma teórica, el tiempo de un bit puede calcularse mediante:

$$t_{bit} = \frac{1}{\text{Baudios}}$$

Este valor permite calcular cuánto debería durar aproximadamente cada bit durante la transmisión. En la práctica, este tiempo puede medirse experimentalmente y compararse con el valor teórico. [3]

5. **Raspberry Pi Pico 2W:** La Raspberry Pi Pico es una tarjeta basada en un microcontrolador que cuenta con diferentes pines de entrada y salida. Entre sus funciones se encuentra la posibilidad de configurar interfaces de comunicación, como UART, para intercambiar información con otros dispositivos.

En esta práctica se utilizó MicroPython para programar las tarjetas Raspberry Pi Pico. Mediante las librerías correspondientes fue posible configurar los pines de transmisión y recepción, establecer los parámetros de UART y desarrollar los programas necesarios para enviar y recibir caracteres y archivos de texto. [4]

En la figura 1 se muestra la distribución de pines de la Raspberry pi pico 2W:

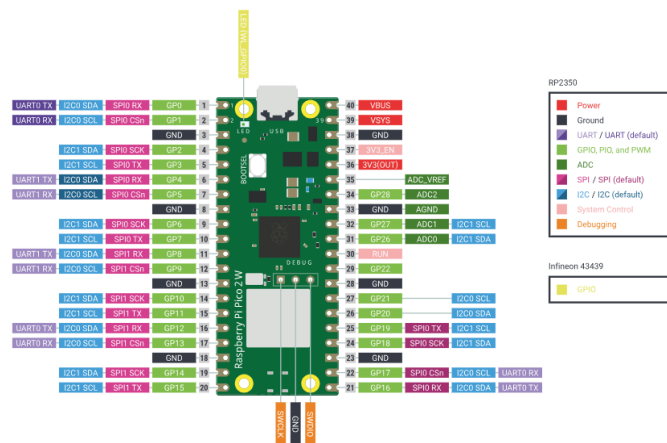


Figura 1. Distribución de pines de la Raspberry pi pico 2W

III. PROCEDIMIENTO

Antes de comenzar con el desarrollo de la práctica, se cargó en la Raspberry Pi Pico 2 W el firmware correspondiente para poder programarla utilizando MicroPython. Posteriormente, se realizaron las conexiones necesarias para observar las señales de transmisión UART en el osciloscopio. Para esto, se conectó el pin 1 (TX) de la Raspberry Pi Pico al canal de medición del osciloscopio y el pin 3 (GND) a la referencia de tierra del instrumento.

1. **Punto 3.3:** Para este punto se utilizó el siguiente programa en MicroPython:

```
import machine
import utime
from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)
uart = UART(0, baudrate=9600, bits=8, parity=0, tx=Pin(0), rx=Pin(1))

while True:
    led.on()
    uart.write("U")
    utime.sleep(1)
    led.off()
    utime.sleep(1)
```

El programa se cargó y ejecutó en la Raspberry Pi Pico 2 W. Este permitió transmitir continuamente el carácter "U" mediante la comunicación UART. Luego, se observó la señal transmitida en el osciloscopio y se realizó la medición del tiempo correspondiente a un bit. El valor obtenido experimentalmente se comparó posteriormente con el valor teórico calculado a partir de la velocidad de transmisión configurada.

2. **Punto 3.4:** Para este punto se utilizó el mismo programa del punto anterior, modificando el carácter transmitido por el carácter "Y". Además, se realizaron diferentes configuraciones de baudios.

Para cada configuración se ejecutó el programa y se capturó la señal correspondiente en el osciloscopio.

Posteriormente, se midió el tiempo de bit de cada señal y se comparó con el valor teórico esperado según los baudios configurados.

3. **Punto 3.5:** En este punto se seleccionaron cinco caracteres ASCII diferentes; O, @, S, ?, W. Para cada uno de ellos se modificaron diferentes parámetros de la comunicación UART, incluyendo:

- ✓ Velocidad de transmisión o baudios.
- ✓ Tipo de paridad.
- ✓ Número de bits de datos

Luego, se ejecutaron los programas correspondientes y se capturaron las señales obtenidas con el osciloscopio.

Además, las tramas de cada carácter fueron representadas manualmente con el fin de identificar sus diferentes partes, tales como:

- ✓ Bit de inicio.
- ✓ Bits de datos.
- ✓ Bit de paridad, cuando correspondía.
- ✓ Bit o bits de parada.

Las configuraciones utilizadas y los resultados obtenidos se presentan en la sección de análisis de resultados.

4. **Punto 3.6:**

- a) Utilizando los mismos caracteres empleados en el punto anterior, se midió el tiempo total de transmisión de cada carácter mediante el osciloscopio. Después, el tiempo experimental obtenido se comparó con el tiempo teórico calculado teniendo en cuenta la velocidad de transmisión y la cantidad total de bits presentes en cada trama.
- b) Para tres de los caracteres utilizados anteriormente se modificó la configuración de paridad. Posteriormente, se observaron y compararon las nuevas tramas obtenidas en el osciloscopio. Además, las tramas fueron dibujadas manualmente para identificar las diferencias producidas por el cambio en el tipo de paridad.
- c) Finalmente, para los mismos tres caracteres se modificó nuevamente la configuración, esta vez utilizando la opción sin paridad. Las señales obtenidas fueron observadas y comparadas con las tramas anteriores, con el objetivo de identificar los cambios producidos al eliminar el bit de paridad.

5. **Punto 3.7:**

- a) Se modificó el programa para transmitir la siguiente secuencia de 60 caracteres:

ABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890A
BCDEFGHIJKLMNOPQRSTUVWXYZ

La comunicación UART se configuró con los siguientes parámetros:

- ✓ Baudrate: 600 baudios.
- ✓ Bits de datos: 8.
- ✓ Paridad: par.

Luego, se observó y capturó la señal completa en el osciloscopio. Se midió el tiempo total de transmisión de la trama y posteriormente se comparó el valor experimental con el valor teórico.

- b) Se realizó nuevamente el envío de la misma secuencia de caracteres, manteniendo los parámetros anteriores, pero

eliminando el bit de paridad. Posteriormente, se capturó la señal en el osciloscopio y se midió el tiempo total de transmisión. Finalmente, el valor obtenido se comparó con el valor teórico correspondiente a esta nueva configuración.

- c) Para este punto se modificó el programa para transmitir el siguiente mensaje:

UMNG LIDER EN INGENIERIA EN
TELECOMUNICACIONES

La comunicación UART se configuró con los siguientes parámetros:

- ✓ Baudrate: 57600 baudios.
- ✓ Bits de datos: 7.
- ✓ Paridad: par.
- ✓ Bits de parada: 2.

Después, se observó y capturó la señal generada en el osciloscopio. Se realizó la medición del tiempo total de transmisión de la trama y el resultado experimental se comparó posteriormente con el valor teórico.

6. **Punto 4:**

- a) Para este punto se utilizaron dos microcontroladores: una Raspberry Pi Pico 2 W y una Raspberry Pi Pico. Una de ellas se configuró como transmisor y la otra como receptor. Las conexiones para establecer la comunicación UART se realizaron de la siguiente manera:

Raspberry Pi Pico 2W - TX	Raspberry Pi Pico - RX
GP0/ pin 1 (Tx)	GP1/ pin 2 (RX)
GP1/ pin 2 (RX)	GP0/ pin 1 (TX)
GND / pin 3	GND / pin 3

Tabla 1. Conexión entre los dos microcontroladores.

De esta forma, el pin de transmisión de un dispositivo se conectó al pin de recepción del otro, y ambos microcontroladores compartieron una conexión común de tierra.

- b) Se diseñaron dos programas en MicroPython denominados TX.py y RX.py, correspondientes al transmisor y receptor, respectivamente. En el programa del transmisor se utilizó la instrucción `input()` para solicitar al usuario, por medio de la consola, el carácter que deseaba enviar. El dato fue ingresado directamente desde la consola y posteriormente transmitido mediante UART.

Por su parte, el programa receptor permaneció esperando la llegada de información. Al recibir y decodificar el carácter enviado, este se mostró en la consola del dispositivo receptor y se activó un LED indicador durante cinco segundos. Después de recibir correctamente el carácter, el receptor envió un carácter de confirmación o ACK de regreso al transmisor.

Cuando el transmisor recibió la confirmación, se hizo parpadear el LED durante tres segundos para indicar que la comunicación se había realizado correctamente. Finalmente, el proceso se repitió, permitiendo al usuario ingresar nuevos caracteres para realizar más transmisiones.

- c) Para esta parte se desarrolló un programa denominado generadortxt.py, ejecutado en el microcontrolador

transmisor. El programa solicitaba al usuario la cantidad de líneas que debía contener el archivo, permitiendo seleccionar un valor entre 1 y 1000. Después de ingresar la cantidad deseada, el programa generaba el archivo alfabetos.txt, en el cual cada línea contenía el alfabeto en inglés y un número consecutivo que permitía identificar su posición.

Luego, se desarrolló el programa enviar.py, el cual solicitaba al usuario el intervalo de tiempo entre el envío de cada línea. Una vez ingresado este valor, el programa leía el archivo alfabetos.txt y realizaba la transmisión de la información línea por línea mediante UART.

En el microcontrolador receptor se desarrolló un programa encargado de recibir la información transmitida. Las líneas recibidas fueron procesadas y almacenadas en un archivo denominado recibido.txt.

Finalmente, se realizaron diferentes pruebas modificando la cantidad de líneas del archivo y el intervalo de tiempo entre cada envío. Durante estas pruebas se registraron los tiempos de transmisión y recepción, así como la cantidad de líneas enviadas y recibidas. Esto permitió analizar el comportamiento de la transferencia cuando se utilizaban diferentes cantidades de información y distintos intervalos de envío.

IV. ANÁLISIS DE RESULTADOS

- Punto 3.3:** Para este punto se configuró la comunicación UART con una velocidad de transmisión de 9600 baudios, 8 bits de datos y paridad par. Posteriormente, se transmitió de forma repetitiva el carácter “U” y se observó la señal generada mediante el osciloscopio. Para obtener una señal estable, se configuró adecuadamente el disparo o *trigger* del osciloscopio.

En la figura 2 se muestra la señal obtenida en el osciloscopio:

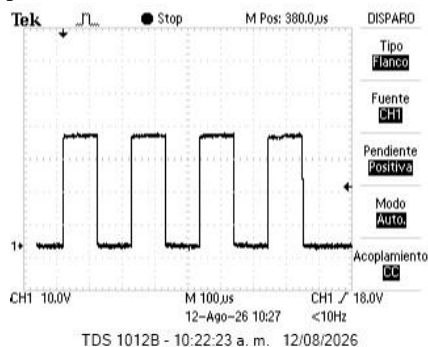


Figura 2. Señal del carácter U a 9600 baudios.

Se calcula el tiempo de bit teórico:

$$t_b = \frac{1}{9600} = 104.16\mu s$$

En la figura 3 se muestra el tiempo de bit experimental medido con los cursores del osciloscopio.

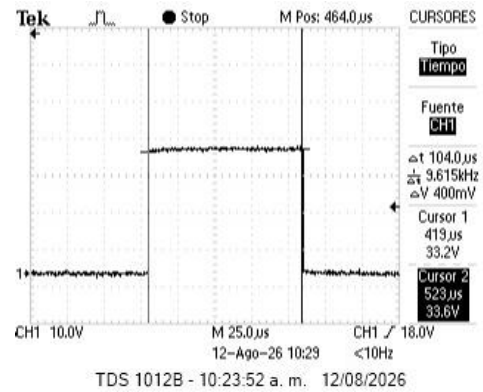


Figura 3. Tiempo de bit del carácter U a 9600 baudios.

Se calculó el % de error entre el tiempo de bit teórico y el experimental:

$$\%Error = \frac{104\mu s - 104.16\mu s}{104.16\mu s} = 0.0015\%$$

El porcentaje de error es muy bajo, por lo que se puede concluir que el tiempo de bit medido experimentalmente es muy cercano al valor teórico esperado para una comunicación configurada a 9600 baudios. Las pequeñas diferencias pueden deberse principalmente a la precisión de la medición realizada con los cursores del osciloscopio y a la resolución de la escala utilizada.

2. Punto 3.4:

Para este punto se transmitió el carácter “Y” utilizando diferentes velocidades de transmisión o baudios. Para cada configuración se observó la señal UART en el osciloscopio y se midió experimentalmente el tiempo de duración de un bit. Posteriormente, estos valores se compararon con los tiempos teóricos calculados.

En la figura 4 y 5 se muestran los valores obtenidos experimentalmente para 1200 baudios:

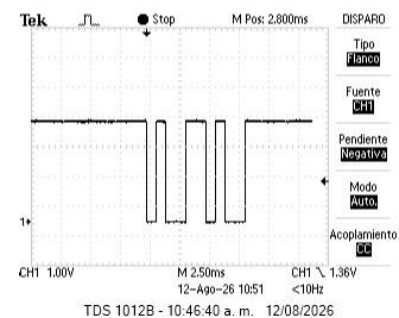


Figura 4. Señal del carácter Y a 1200 baudios.

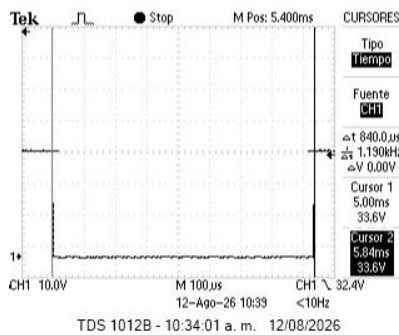


Figura 5. Tiempo de bit del carácter Y a 1200 baudios.

En la figura 6 y 7 se muestran los valores obtenidos experimentalmente para 4800 baudios:

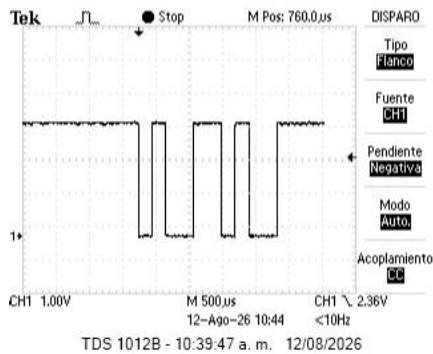


Figura 6. Señal del carácter Y a 4800 baudios.

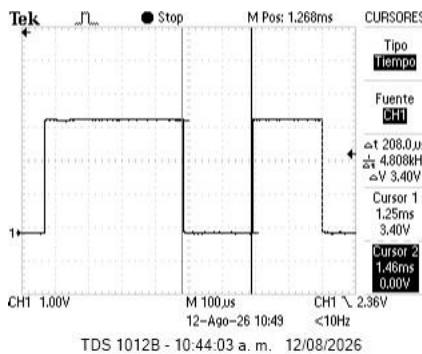


Figura 7. Tiempo del bit del carácter Y a 4800 baudios.

En la figura 8 se muestra la señal obtenida experimentalmente para 9600 baudios:

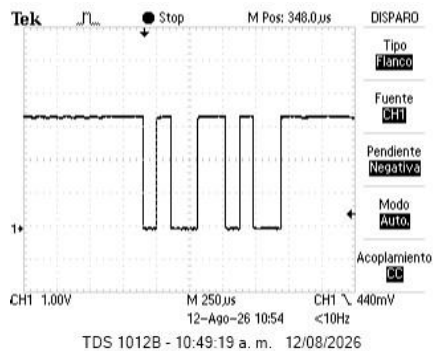


Figura 8. Señal del carácter Y a 9600 baudios.

En la figura 9 y 10 se muestran los valores obtenidos experimentalmente para 38400 baudios:

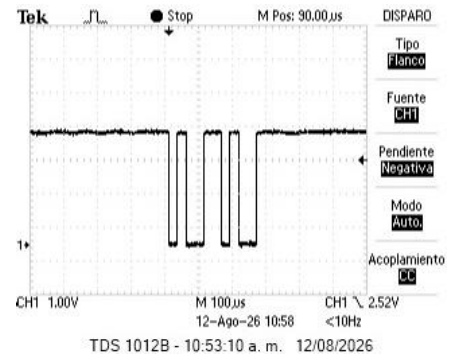


Figura 9. Señal del carácter Y a 38400 baudios.

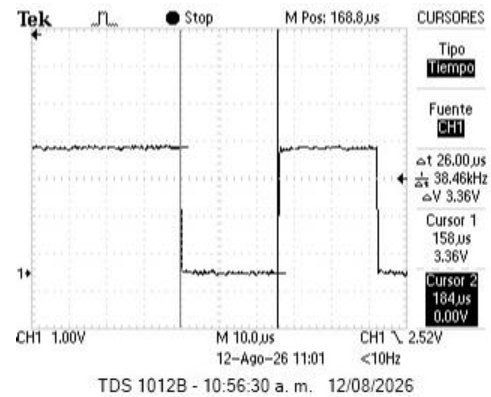


Figura 10. Tiempo de bit del carácter Y a 38400 baudios.

En la figura 11 y 12 se muestran los valores obtenidos experimentalmente para 57600 baudios:

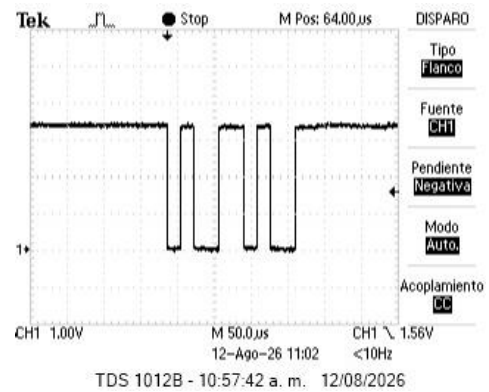


Figura 11. Señal del carácter Y a 57600 baudios.

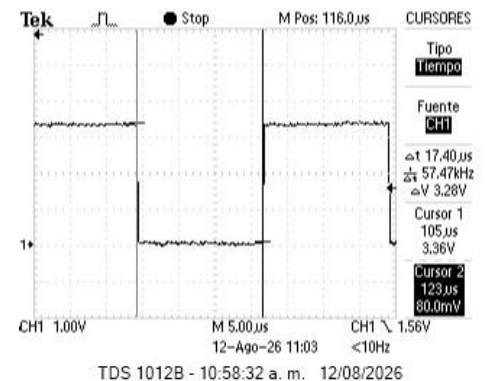


Figura 12. Tiempo del bit del carácter Y a 57600 baudios.

En la figura 13 y 14 se muestran los valores obtenidos experimentalmente para 115200 baudios:

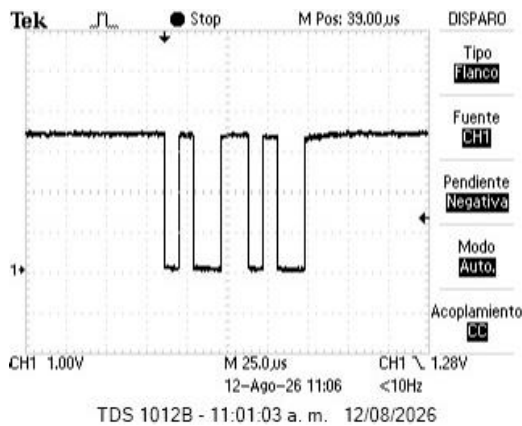


Figura 13. Señal del carácter Y a 115200 baudios.

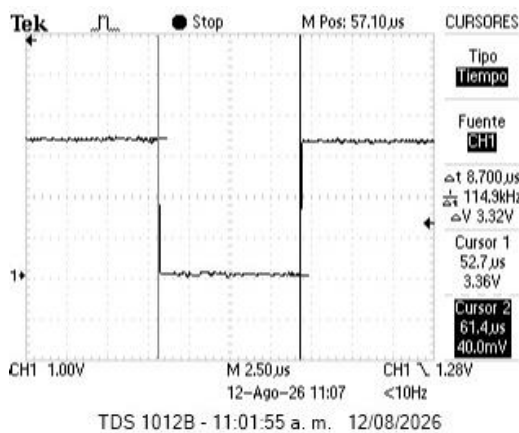


Figura 14. Tiempo de bit del carácter Y a 115200 baudios.

En la tabla 2, se resume las mediciones obtenidas.

Baudios	Tiempo de bit (Teórico)	Tiempo de bit (experimental)	Δt	% error	Voltaje máximo	Voltaje mínimo
1200	833.33 μs	840 μs	6.67 μs	0.8%	3.4 V	0V
4800	208.3 μs	208 μs	0.3 μs	0.14%	3.4 V	0V
9600	104.1 μs	104 μs	0.1 μs	0.09%	3.4 V	0V
38400	26.04 μs	26 μs	0.04 μs	0.15%	3.4 V	0V
57600	17.36 μs	17.40 μs	0.04 μs	0.23%	3.36 V	0V
115200	8.68 μs	8.70 μs	0.02 μs	0.23%	3.32 V	0V

Tabla 2. Resumen de medidas experimentales de tiempo de bit.

Las pequeñas diferencias entre los valores teóricos y experimentales pueden deberse principalmente a la resolución del osciloscopio y a la precisión al realizar la medición del tiempo utilizando los cursores. Sin embargo, debido a que todos los porcentajes de error fueron bajos, se puede concluir que la velocidad de transmisión configurada en la Raspberry Pi Pico funcionó de acuerdo con los valores esperados.

3. Punto 3.5:

En este punto se analizaron cinco caracteres ASCII diferentes: O, @, S, ? y W. Para cada carácter se identificó su valor decimal y su representación en código binario. Luego, se observó la señal generada en el osciloscopio y se realizó su representación

para identificar las diferentes partes de la trama UART, incluyendo el bit de inicio, los bits de datos, el bit de paridad y el bit de parada.

Además, se tuvo en cuenta que, aunque el valor binario normalmente se escribe desde el bit más significativo (MSB) hasta el bit menos significativo (LSB), en la comunicación UART los bits de datos se transmiten comenzando por el LSB. Esto permitió comparar la información teórica de cada carácter con la señal observada experimentalmente.

En la tabla 3 se muestran los parámetros de cada carácter.

ASCII	Decimal	Baudios	Bits	Paridad	Dibujo
O	01001111	9600	8	Ninguna	Figura 16
@	01000000	4800	8	Par	Figura 18
S	01010011	38400	8	Impar	Figura 20
?	00111111	57600	7	Ninguna	Figura 22
W	01010111	115200	7	Par	Figura 24

Tabla 3. Parámetros para los 5 caracteres.

En la figura 15 y 16 se muestra la señal obtenida en el osciloscopio para el carácter “O” y la señal dibujada.

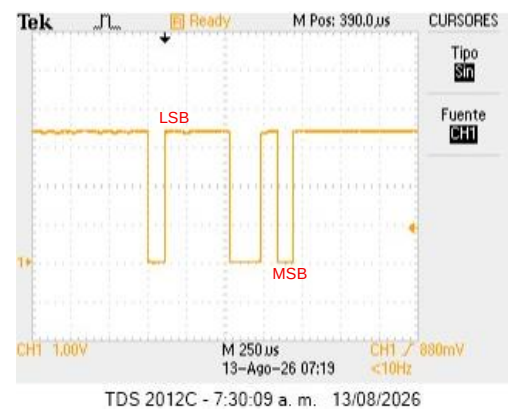


Figura 15. Señal del carácter O en el osciloscopio.

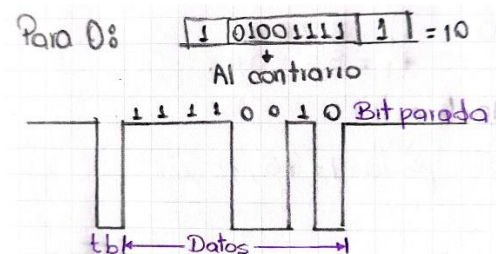


Figura 16. Señal del carácter O dibujada.

En la figura 17 y 18 se muestra la señal obtenida en el osciloscopio para el carácter “@” y la señal dibujada.

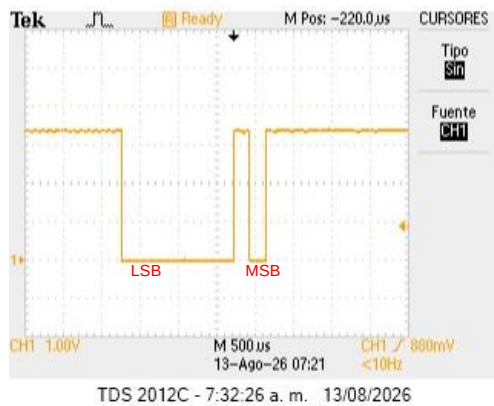


Figura 17. Señal del carácter @ en el osciloscopio

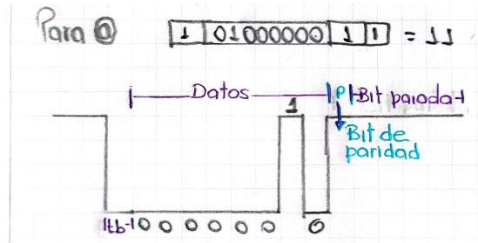


Figura 18. Señal del carácter @ dibujada.

En la figura 19 y 20 se muestra la señal obtenida en el osciloscopio para el carácter “S” y la señal dibujada.

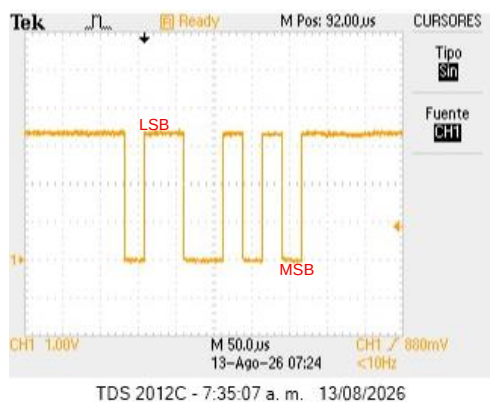


Figura 19. Señal del carácter S en el osciloscopio.

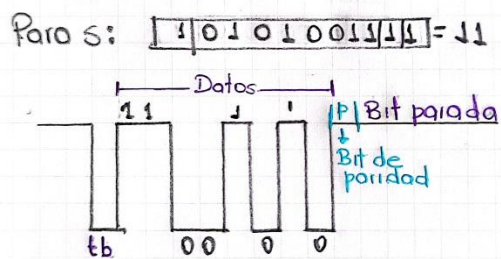


Figura 20. Señal del carácter S dibujada.

En la figura 21 y 22 se muestra la señal obtenida en el osciloscopio para el carácter “?” y la señal dibujada.



Figura 21. Señal del carácter ? en el osciloscopio.

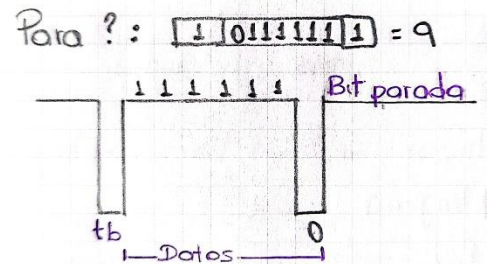


Figura 22. Señal del carácter ? dibujada.

En la figura 23 y 24 se muestra la señal obtenida en el osciloscopio para el carácter “W” y la señal dibujada.

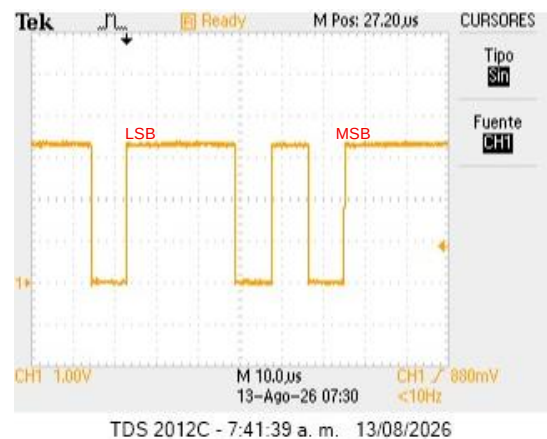


Figura 23. Señal del carácter W en el osciloscopio.

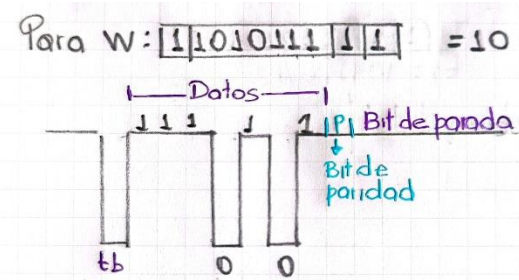


Figura 24. Señal del carácter W dibujada.

A partir de las señales obtenidas en el osciloscopio y de las tramas dibujadas manualmente, se pudo comprobar que los resultados experimentales coincidieron con la información teórica de cada carácter ASCII transmitido. En las diferentes

señales fue posible identificar correctamente el bit de inicio, los bits de datos, el bit de paridad y el bit de parada.

4. Punto 3.6:

a)

En este punto se midió el tiempo total requerido para transmitir cada uno de los caracteres utilizados en el punto 3.5. Para realizar la comparación, se tuvo en cuenta la configuración UART utilizada para cada carácter, especialmente los baudios, el número de bits de datos, la paridad y los bits de parada.

Para la O:

Se calcula el tiempo de bit teórico:

$$t_b = \frac{1}{9600} = 104.1\mu s$$

Se calcula el tiempo total de la trama, en la figura 16 se observa que en total se envían 10 bits:

$$T_{trama} = 104.1\mu s * 10 = 1.041ms$$

En la figura 25 se observa el tiempo total medido para el carácter "O" en el osciloscopio.

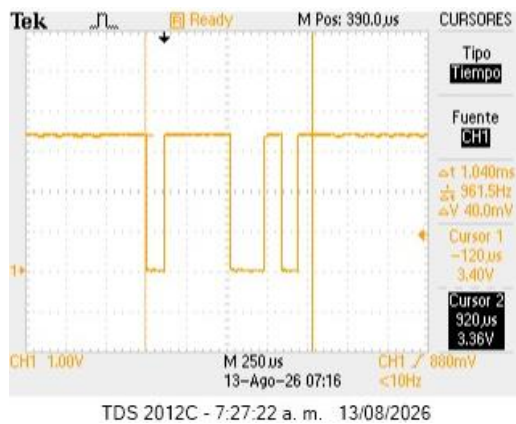


Figura 25. Tiempo total del carácter O en el osciloscopio.

Se calcula el error entre el valor teórico y el experimental:

$$\%Error = \frac{1.041ms - 1.040ms}{1.041ms} = 0.09\%$$

Para el @:

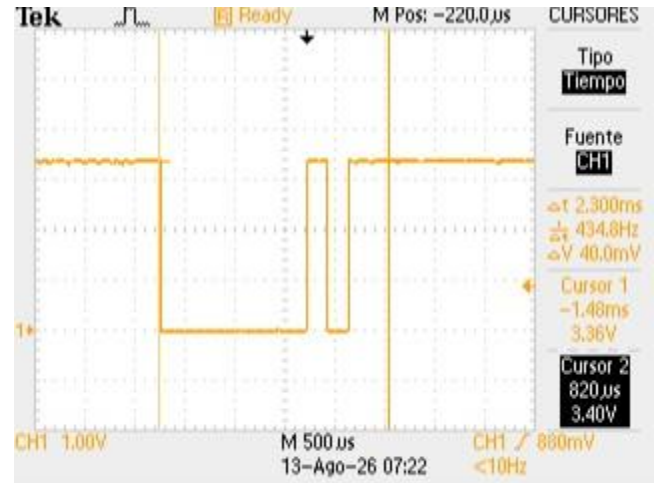
Se calcula el tiempo de bit teórico:

$$t_b = \frac{1}{4800} = 208.3\mu s$$

Se calcula el tiempo total de la trama, en la figura 18 se observa que en total se envían 11 bits:

$$T_{trama} = 208.3\mu s * 11 = 2.29ms$$

En la figura 26 se observa el tiempo total medido para el carácter "@" en el osciloscopio.



TDS 2012C - 7:33:46 a. m. 13/08/2026

Figura 26. Tiempo total del carácter @ en el osciloscopio.

Se calcula el error entre el valor teórico y el experimental:

$$\%Error = \frac{2.29ms - 2.30ms}{2.29ms} = 0.4\%$$

Para la S:

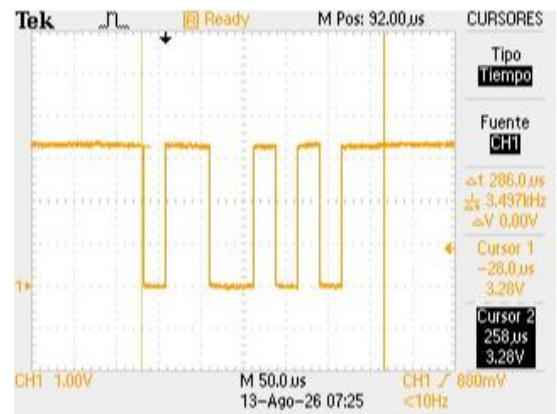
Se calcula el tiempo de bit teórico:

$$t_b = \frac{1}{38400} = 26.04\mu s$$

Se calcula el tiempo total de la trama, en la figura 20 se observa que en total se envían 11 bits:

$$T_{trama} = 26.04\mu s * 11 = 286.4\mu s$$

En la figura 27 se observa el tiempo total medido para el carácter "S" en el osciloscopio.



TDS 2012C - 7:36:42 a. m. 13/08/2026

Figura 27. Tiempo total del carácter S en el osciloscopio.

Se calcula el error entre el valor teórico y el experimental:

$$\%Error = \frac{286.4\mu s - 286\mu s}{286.4\mu s} = 0.13\%$$

Para la ?:

Se calcula el tiempo de bit teórico:

$$t_b = \frac{1}{57600} = 17.36\mu s$$

Se calcula el tiempo total de la trama, en la figura 22 se observa que en total se envían 9 bits:

$$T_{trama} = 17.36\mu s * 9 = 156.24\mu s$$

En la figura 28 se observa el tiempo total medido para el carácter “?” en el osciloscopio.

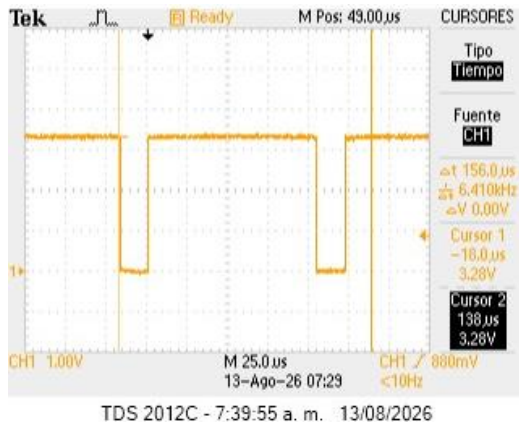


Figura 28. Tiempo total del carácter ? en el osciloscopio.

Se calcula el error entre el valor teórico y el experimental:

$$\%Error = \frac{156.2\mu s - 156\mu s}{156\mu s} = 0.12\%$$

Para la W:

Se calcula el tiempo de bit teórico:

$$t_b = \frac{1}{115200} = 8.68\mu s$$

Se calcula el tiempo total de la trama, en la figura 24 se observa que en total se envían 10 bits:

$$T_{trama} = 8.68\mu s * 10 = 86.8\mu s$$

En la figura 29 se observa el tiempo total medido para el carácter “W” en el osciloscopio.

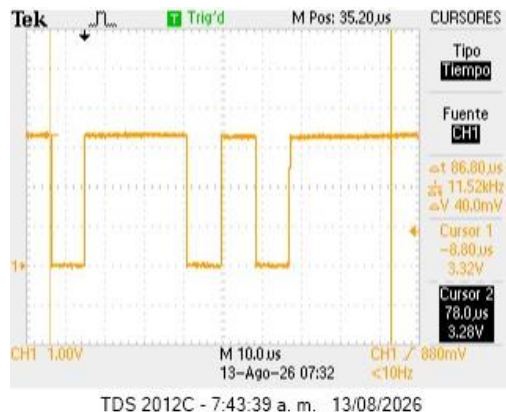


Figura 29. Tiempo total del carácter W en el osciloscopio.

Se calcula el error entre el valor teórico y el experimental:

$$\%Error = \frac{86.8\mu s - 86.8\mu s}{86.8\mu s} = 0\%$$

Al comparar los tiempos totales de transmisión obtenidos experimentalmente con los valores calculados teóricamente, se observó que los resultados fueron muy cercanos entre sí. Estos resultados permiten comprobar que el tiempo total de transmisión depende directamente de la cantidad de bits que forman la trama y de los baudios configurado.

b)

En este punto se modificó el tipo de paridad de los caracteres @, S y W, con el objetivo de observar cómo este cambio afecta la estructura de la trama UART. Para cada carácter se mantuvieron los demás parámetros de comunicación y únicamente se modificó la configuración de paridad.

Luego, las señales fueron observadas y capturadas utilizando el osciloscopio. Además, se realizaron los dibujos correspondientes para identificar y comparar el bit de inicio, los bits de datos, el bit de paridad y el bit de parada.

Para el @:

Se cambió la paridad de par a impar, en la figura 30 se muestra la señal obtenida en el osciloscopio y el tiempo total medido.

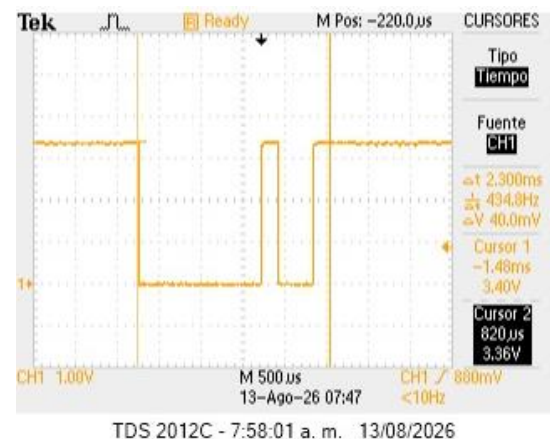


Figura 30. Tiempo total del carácter @ con paridad impar.

En la figura 31 se muestra la señal del carácter “@” dibujada con paridad impar.

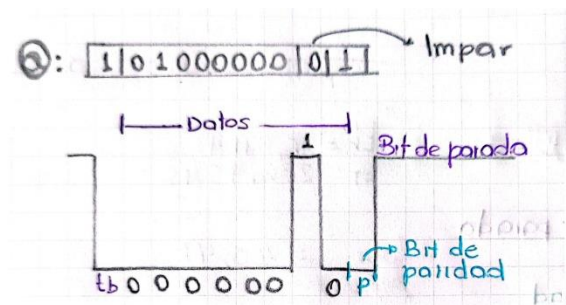


Figura 31. Señal dibujada del carácter @ con paridad impar.

Para el carácter “@”, se compararon dos tramas UART utilizando diferentes configuraciones de paridad. En la figura 18 se utilizó paridad par, mientras que en la figura 31 se utilizó paridad impar.

Al comparar las dos señales se observa que los bits de inicio, los bits de datos y el bit de parada permanecen iguales, ya que el carácter transmitido no cambió. La diferencia se encuentra en el bit de paridad. Con paridad par, el bit de paridad toma el valor 1, mientras que con paridad impar toma el valor 0. Este cambio permite que la cantidad total de unos de la trama cumpla con el tipo de paridad seleccionado.

Para la S:

Se cambió la paridad de impar a par, en la figura 32 se muestra la señal obtenida en el osciloscopio y el tiempo total medido.

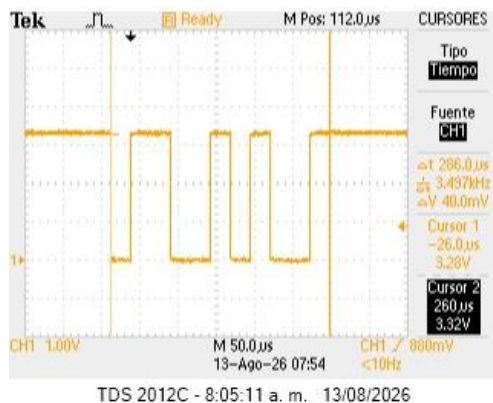


Figura 32. Tiempo total del carácter S con paridad par.

En la figura 33 se muestra la señal del carácter “S” dibujada con paridad par.

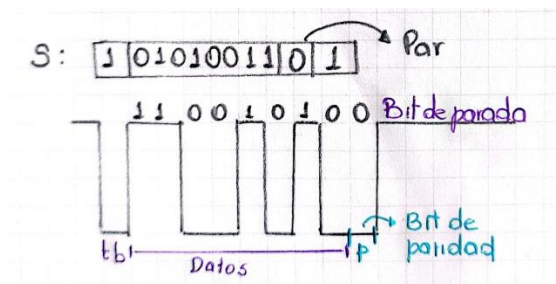


Figura 33. Señal dibujada del carácter S con paridad par.

Para el carácter S se realizaron dos pruebas modificando el tipo de paridad. En la primera la figura 20 se utilizó paridad impar, mientras que en la figura 33 se utilizó paridad par.

Al comparar las dos tramas se observa que el bit de inicio y los bits de datos permanecen iguales en ambas señales, debido a que el carácter transmitido no cambia. La diferencia se presenta en el bit de paridad. En la configuración de paridad impar, el bit de paridad toma el valor necesario para que la cantidad total de unos sea impar. Al cambiar a paridad par, este bit cambia para que la cantidad total de unos sea par.

En las señales obtenidas se pudo identificar correctamente el bit de inicio, los bits de datos, el bit de paridad y el bit de parada. Esto permite comprobar que el cambio de paridad afecta únicamente el bit destinado a esta función, mientras que los bits que representan el carácter permanecen sin cambios.

Para la W:

Se cambió la paridad de par a impar, en la figura 34 se muestra la señal obtenida en el osciloscopio y el tiempo total medido.

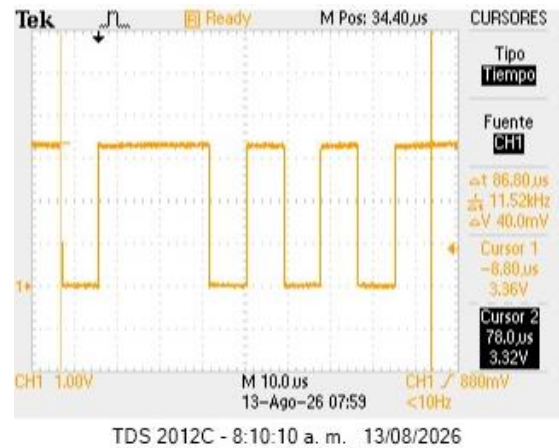


Figura 34. Tiempo total del carácter W con paridad impar.

En la figura 35 se muestra la señal del carácter “W” dibujada con paridad impar.

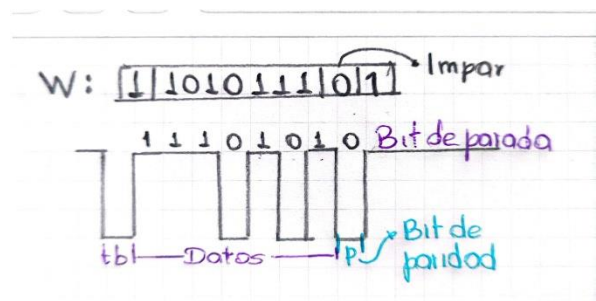


Figura 35. Señal dibujada del carácter W con paridad impar.

Para el carácter W se realizaron dos pruebas modificando el tipo de paridad. En la figura 24 se utilizó paridad par, mientras que en la figura 35 se utilizó paridad impar.

Al comparar las dos tramas se observa que el bit de inicio y los bits de datos permanecen iguales en ambas configuraciones, ya que el carácter transmitido no cambia. La diferencia se presenta en el bit de paridad. Para la configuración de paridad par, este bit toma el valor necesario para que la cantidad total de unos sea par. Al cambiar a paridad impar, el bit de paridad cambia para que la cantidad total de unos sea impar.

De esta manera, se comprobó mediante las señales observadas que el cambio de paridad modifica únicamente el bit destinado a la comprobación de errores, mientras que la información correspondiente al carácter permanece igual.

c)

Para este punto se modificó nuevamente la configuración de UART de los caracteres @, S y W, eliminando el bit de paridad. Posteriormente, se observaron las nuevas señales en el osciloscopio y se compararon con las tramas obtenidas anteriormente cuando se utilizaba paridad.

Para el @:

Se quitó la paridad, por lo que se calculó nuevamente el tiempo total de la trama. Ahora la trama envía 10 bits:

$$T_{trama} = 208.3\mu s * 10 = 2.083ms$$

En la figura 36 se muestra la señal obtenida en el osciloscopio y el tiempo total medido.

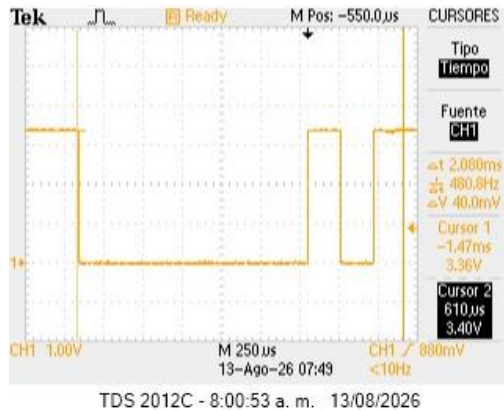


Figura 36. Tiempo total del carácter @ sin paridad.

En la figura 37 se muestra la señal del carácter “@” dibujada sin paridad.

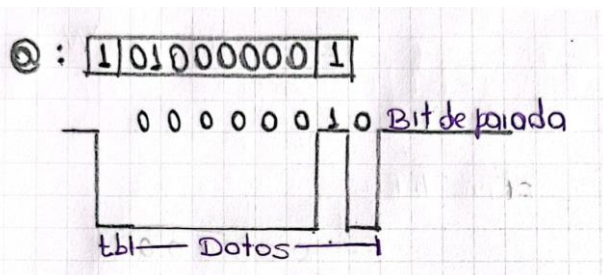


Figura 37. Señal dibujada del carácter @ sin paridad.

Al eliminar la paridad, la trama quedó conformada por 10 bits en total: un bit de inicio, ocho bits de datos y un bit de parada. Al medir el tiempo total de la trama en el osciloscopio, se obtuvo un valor experimental igual al valor teórico calculado.

Al comparar esta trama con la obtenida en el punto 3.6 b), donde se utilizaba paridad, se observa que los bits de inicio y de datos permanecen iguales. La principal diferencia es que en la trama sin paridad desaparece el bit adicional de paridad y el bit de parada aparece inmediatamente después del último bit de datos.

Por lo tanto, al eliminar el bit de paridad la trama pasa de 11 bits a 10 bits, reduciendo el tiempo total necesario para transmitir el carácter.

Para la S:

Se quitó la paridad, por lo que se calculó nuevamente el tiempo total de la trama. Ahora la trama envía 10 bits:

$$T_{trama} = 26.04\mu s * 10 = 260.4\mu s$$

En la figura 38 se muestra la señal obtenida en el osciloscopio y el tiempo total medido.

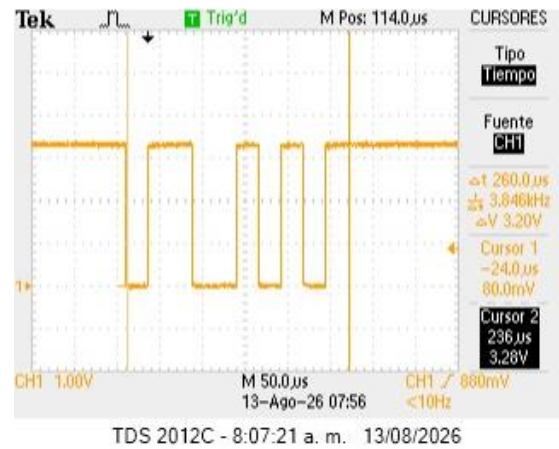


Figura 38. Tiempo total del carácter S sin paridad.

En la figura 39 se muestra la señal del carácter “S” dibujada sin paridad.

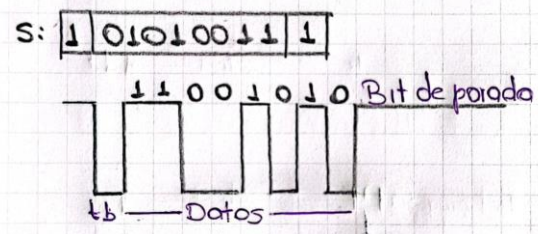


Figura 39. Señal dibujada del carácter S sin paridad.

Para el carácter S se eliminó el bit de paridad utilizado en las pruebas anteriores y se volvió a calcular el tiempo total de la trama. Al trabajar con 8 bits de datos, un bit de inicio y un bit de parada, la nueva trama quedó conformada por 10 bits en total.

Al comparar la gráfica realizada para la transmisión sin paridad con las gráficas del punto 3.6 b), se observa que los bits de inicio y los bits de datos mantienen exactamente la misma secuencia. La diferencia principal se encuentra después de los ocho bits de datos, ya que en las tramas anteriores se encontraba el bit de paridad y, en esta nueva configuración, este bit desaparece y se pasa directamente al bit de parada.

La eliminación del bit de paridad hace que la trama pase de 11 bits a 10 bits, por lo que el tiempo total de transmisión también disminuye.

Para la W:

Se quitó la paridad, por lo que se calculó nuevamente el tiempo total de la trama. Ahora la trama envía 9 bits:

$$T_{trama} = 8.68\mu s * 9 = 78.12\mu s$$

En la figura 40 se muestra la señal obtenida en el osciloscopio y el tiempo total medido.

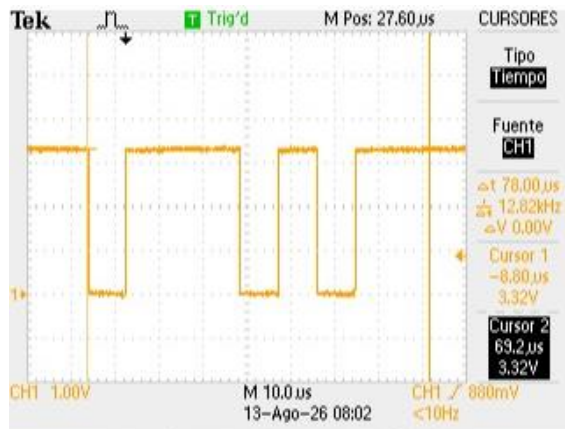


Figura 40. Tiempo total del carácter W sin paridad.

En la figura 41 se muestra la señal del carácter “W” dibujada sin paridad.

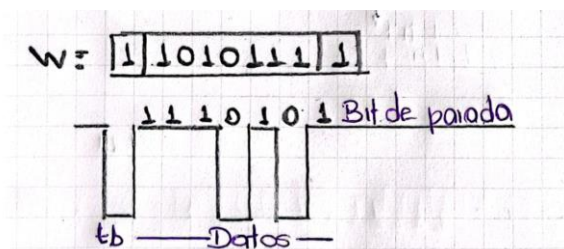


Figura 41. Señal dibujada del carácter W sin paridad.

Al comparar esta gráfica con las obtenidas anteriormente en el punto 3.6 b), se observa que los bits de inicio y los bits de datos mantienen la misma secuencia. La principal diferencia es que en la nueva configuración se eliminó el bit de paridad, por lo que después de los siete bits de datos se encuentra directamente el bit de parada.

Por esta razón, la trama pasó de 10 bits con paridad a 9 bits sin paridad, reduciendo el tiempo total necesario para transmitir el carácter.

5. Punto 3.7:

a)

Para esta prueba se transmitió una cadena de 60 caracteres utilizando una velocidad de 600 baudios, 8 bits de datos, paridad par y 1 bit de parada.

Para este punto, utilizamos la siguiente cadena de caracteres:

ABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890ABCDE
FGHIJKLMNOPQRSTUVWXYZ

La señal correspondiente a toda la transmisión fue observada y capturada mediante el osciloscopio para determinar experimentalmente el tiempo total de la trama.

Para calcular el tiempo teórico se tuvo en cuenta que cada carácter está compuesto por un bit de inicio, ocho bits de datos, un bit de paridad y un bit de parada:

Entonces: $1 + 8 + 1 + 1 = 11 \text{ bits}$

Como son 60 caracteres: $60 * 11 = 660 \text{ bits}$

Se calcula el tiempo de bit:

$$t_b = \frac{660}{600} = 1.1s$$

En la figura 42 se muestra el tiempo total de la trama en el osciloscopio.

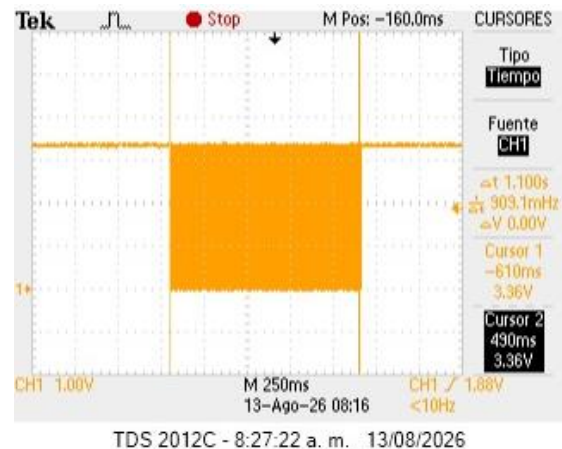


Figura 42. Tiempo total de la trama de 60 caracteres en el osciloscopio.

En la medición realizada con el osciloscopio se obtuvo un tiempo experimental de 1.100 s, este resultado coincide con el tiempo teórico calculado. Por lo tanto, se comprobó que el tiempo medido experimentalmente corresponde al tiempo esperado para la transmisión de los 60 caracteres bajo la configuración establecida.

b)

Para este punto se realizó nuevamente la transmisión de los 60 caracteres utilizados en el punto anterior, manteniendo una velocidad de 600 baudios y 8 bits de datos, pero eliminando el bit de paridad.

Al no utilizar paridad, cada carácter queda conformado por un bit de inicio, ocho bits de datos y un bit de parada:

Entonces: $1 + 8 + 1 = 10 \text{ bits}$

Como son 60 caracteres: $60 * 10 = 600 \text{ bits}$

Se calcula el tiempo de bit:

$$t_b = \frac{600}{600} = 1s$$

En la figura 43 se muestra el tiempo total de la trama en el osciloscopio.



Figura 43. Tiempo total de la trama de 60 caracteres sin paridad.

Al comparar este resultado con el punto 3.7 a), se observa que al eliminar el bit de paridad cada carácter pasa de tener 11 bits a 10 bits. Como consecuencia, para los 60 caracteres transmitidos se pasa de 660 bits a 600 bits, reduciendo el tiempo teórico de transmisión de 1.1 s a 1.0 s. La medición realizada en el osciloscopio confirmó esta reducción y coincidió con el valor teórico esperado.

c)

Para esta prueba se transmitió el mensaje:

“UMNG LIDER EN INGENIERIA EN TELECOMUNICACIONES”

Utilizando una velocidad de 57600 baudios, 7 bits de datos, paridad par y 2 bits de parada.

El mensaje contiene 45 caracteres, incluyendo los espacios.

Cada carácter tiene un bit de inicio, 7 bits de datos, 1 bit de paridad y 2 de parada, entonces:
 $1 + 7 + 1 + 2 = 11 \text{ bits}$

Como son 45 caracteres: $45 * 11 = 495 \text{ bits}$

Se calcula el tiempo de bit:

$$t_b = \frac{495}{57600} = 8.6ms$$

En la figura 44 se muestra el tiempo total de la trama en el osciloscopio.



Figura 44. Tiempo total de la trama de 45 caracteres.

El resultado permite comprobar que la cantidad de bits de cada carácter y la velocidad de transmisión determinan directamente el tiempo total necesario para enviar el mensaje. Además, se verificó experimentalmente el efecto de utilizar 7 bits de datos, un bit de paridad y dos bits de parada en la duración de la trama.

6. Punto 4: RETO DE PROGRAMACIÓN

a)

Interconexión de Hardware: Interconecte dos dispositivos Raspberry Pi Pico cruzando las líneas de transmisión y recepción (el pin TX del dispositivo 1 al pin RX del dispositivo 2, y viceversa). Es estrictamente necesario unificar la señal GND entre ambos circuitos para establecer un nivel de referencia de voltaje común. Adicionalmente, integre una pantalla (por ejemplo, un display OLED SSD1306 o LCD 16x2 vía I2C) en el circuito para la visualización local de los datos. Conecte los dos canales del osciloscopio digital a las líneas de datos TX y RX.

La interconexión entre las dos placas se realizó de la siguiente manera:

Raspberry Pi Pico 2 W – TX	Raspberry Pi Pico – RX
GP0 / pin 1 (TX)	GP1 / pin 2 (RX)
GP1 / pin 2 (RX)	GP0 / pin 1 (TX)
GND / pin 3	GND / pin 3

La conexión de transmisión y recepción se realizó de forma cruzada: el pin TX de la Pico 2 W se conectó al pin RX de la Pico receptora, y el pin RX de la Pico 2 W se conectó al pin TX de la Pico receptora. Además, ambas placas compartieron la misma referencia de tierra (GND). Cada placa fue alimentada mediante su propia conexión USB.

La pantalla OLED se conectó a la Raspberry Pi Pico utilizada como receptor:

OLED	Raspberry Pi Pico – RX
VDD	3V3 OUT / pin 36
GND	GND / pin 38
SDA	GP4 / pin 6
SCL/SCK	GP5 / pin 7

La interfaz I2C del OLED se configuró utilizando GP4 como SDA y GP5 como SCL. La pantalla se empleó para mostrar localmente el carácter recibido por el microcontrolador RX.

b)

Transmisión y recepción de caracteres (Half-Duplex): Diseñe un programa de transmisión (TX.py) y uno de recepción (RX.py) para ejecutarse en los microcontroladores bajo la siguiente lógica de operación:

– Transmisor (TX.py): Utilice la instrucción input() en la consola para preguntar al usuario el carácter o código específico que desea enviar. El sistema debe capturar el dato exclusivamente por esta vía; no lo debe leer mediante estados lógicos en los pines físicos del dispositivo.

– Receptor (RX.py): Al detectar y decodificar el carácter, el dispositivo debe imprimir el carácter recibido en la pantalla local, y simultáneamente encender un LED indicador durante cinco segundos. Inmediatamente después, debe enviar un carácter de confirmación (ACK) de vuelta al transmisor.

– Confirmación: Cuando el dispositivo transmisor reciba el carácter de respuesta, encenderá un LED de forma intermitente (parpadeo) durante tres segundos, indicando el éxito del enlace. El proceso se repite de forma permanente solicitando un nuevo ingreso de datos por consola.

EVIDENCIAS EXPERIMENTALES

Inicialmente, el receptor fue ejecutado correctamente y quedó en estado de espera, mostrando el mensaje correspondiente en el OLED.



Figura 45. Pantalla OLED del receptor en estado de espera

Posteriormente se realizó una prueba enviando el carácter A desde el transmisor. El transmisor reportó el envío del carácter y posteriormente la recepción del ACK.

TX-Consola

```
Consola x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
TRANSMISOR listo
Escriba un caracter:
Caracter: A
Enviado: A
ACK recibido
Caracter:
```

Figura 46. Consola del transmisor.

RX-Consola

```
Consola x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Receptor listo
Recibido: A
```

Figura 47. Consola del receptor.



Figura 48. Pantalla OLED mostrando el carácter recibido.

FUNCIONAMIENTO

La prueba realizada permitió comprobar la siguiente secuencia de comunicación:

1. El usuario introduce el carácter A en la consola del transmisor.
2. La Raspberry Pi Pico 2 W envía el carácter mediante UART.
3. La Raspberry Pi Pico receptora recibe y decodifica el carácter.
4. El carácter recibido se muestra en la consola y en la pantalla OLED.
5. El LED del receptor se activa durante cinco segundos.
6. El receptor envía el carácter K como confirmación (ACK).
7. El transmisor recibe el ACK y reporta la confirmación en consola.

c)

Transmisión de archivos: Cree un programa llamado generadortxt.py para un microcontrolador transmisor (TX) con todos los parámetros de la comunicación configurables. El programa debe preguntar al usuario el número de líneas de alfabetos en inglés que debe crear (entre 1 y 1000). El programa crea el archivo alfabetos.txt y enumera cada una de las líneas.

0001 ABCDE .. XYZ

0002 ABCDE .. XYZ

...

1000 ABCDE .. XYZ

Otro programa llamado enviar.py debe preguntar al usuario el tiempo del intervalo de envío entre líneas. Una vez se reciba la respuesta del usuario, deberá iniciar el envío línea por línea del archivo alfabetos.txt.

En el otro microcontrolador (RX) se deben recibir línea a línea y guardar el resultado en el archivo recibido.txt

Analice varios casos de envíos y de tiempos requeridos para la transferencia. ¿Cómo se pueden mejorar los programas para medir el tiempo empleado en la transferencia completa del archivo?

alfabeto.txt

El programa generadortxt.py solicita al usuario la cantidad de líneas, con un rango entre 1 y 1000, y genera el archivo alfabetos.txt. Cada línea contiene un número de cuatro dígitos seguido por las 26 letras del alfabeto.

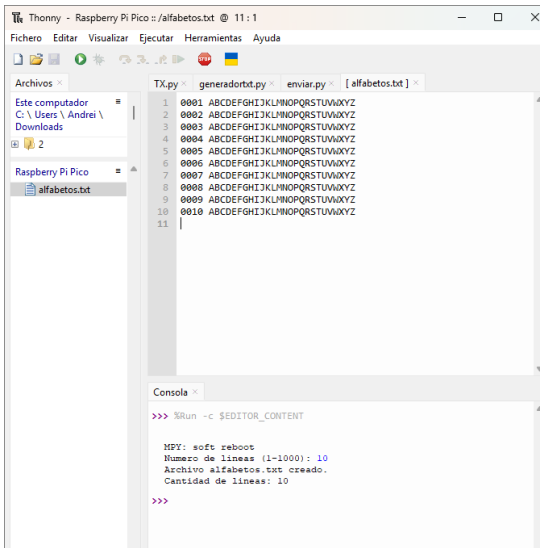


Figura 49. Archivo alfabetos.txt .

Enviar.py

El programa enviar.py lee el archivo alfabetos.txt, solicita al usuario el intervalo entre líneas y realiza la transmisión mediante UART. El tiempo de transmisión se mide utilizando time.time(). Al finalizar las líneas se envía el indicador <<FIN>>.

Recibir.py

El programa recibir.py permanece esperando información por UART. Cuando recibe datos, los almacena temporalmente en un buffer y separa cada línea mediante el carácter de salto de línea. Cada línea recibida se guarda en recibido.txt y se

incrementa el contador de líneas. Cuando se detecta <<FIN>>, se cierra el archivo y se muestra el tiempo total de recepción.

Experimental

Se realizaron 15 pruebas modificando la cantidad de líneas y el intervalo entre líneas. Se utilizaron archivos de 10, 100, 500 y 1000 líneas. Los intervalos utilizados fueron 1 s, 0.1 s, 0.01 s y 0 s. Para cada prueba se registraron las líneas enviadas, recibidas, perdidas y los tiempos medidos en TX y RX.

RESULTADOS DE TRANSMISIÓN DE ARCHIVOS

Líneas	Intervalo (s)	Enviadas	Recibidas	Perdidas	Pérdida (%)
10	1	10	10	0	0
10	0.1	10	10	0	0
10	0.01	10	10	0	0
10	0	10	9	1	10
100	1	100	100	0	0
100	0.1	100	100	0	0
100	0.01	100	12	88	88
100	0	100	9	91	91
500	1	500	500	0	0
500	0.1	500	500	0	0
500	0.01	500	12	488	97.6
500	0	500	9	491	98.2
1000	0.1	1000	1000	0	0
1000	0.01	1000	12	988	98.8
1000	0	1000	9	991	99.1

Se observa que las pérdidas aumentan considerablemente al reducir el intervalo de transmisión. Para un intervalo de 0 s y 1000 líneas se obtuvo una pérdida del 99.1 %.

COMPARACIÓN DE TIEMPOS DE TRANSMISIÓN Y RECEPCIÓN

Líneas	Intervalo (s)	TX (s)	RX (s)	Δt (s)	Dif. (%)
10	1	9.042	9.030	0.012	0.133
10	0.1	0.942	0.926	0.016	1.699
10	0.01	0.342	0.087	0.255	74.561
10	0	0.304	—	—	—
100	1	99.042	99.051	0.009	0.009
100	0.1	9.942	9.946	0.004	0.040
100	0.01	1.556	—	—	—
100	0	0.574	—	—	—
500	1	499.043	499.074	0.031	0.006
500	0.1	49.942	49.952	0.010	0.020
500	0.01	6.756	—	—	—
500	0	1.776	—	—	—
1000	0.1	99.942	99.973	0.031	0.031
1000	0.01	13.254	—	—	—
1000	0	3.276	—	—	—

Permite comparar los tiempos medidos en el transmisor y el receptor. Para los casos de transferencia completa con intervalos de 1 s y 0.1 s, la diferencia temporal fue reducida.

¿Qué pasa si el intervalo entre líneas en el programa enviar.py es cero?

Cuando el intervalo es cero, no se introduce ninguna pausa entre las líneas. El transmisor intenta enviar los datos consecutivamente y el receptor debe procesarlos a la misma velocidad. Debido a la capacidad limitada de procesamiento y recepción, se producen pérdidas cuando la cantidad de datos aumenta.

En la prueba de 1000 líneas con intervalo 0 s se enviaron 1000 líneas, pero solamente se recibieron 9, lo que corresponde a una pérdida del 99.1 %. Esto demuestra que un intervalo cero no es adecuado para una transferencia confiable con la implementación utilizada.

En las pruebas con pérdidas, el receptor tampoco alcanzó a detectar correctamente el indicador <<FIN>>. Por esta razón, recibido.txt puede quedar abierto por el programa y no ser posible abrirlo desde Thonny mientras la transferencia permanece incompleta.

V. CONCLUSIONES

- Se logró comprender de forma práctica el funcionamiento de la comunicación UART, identificando los diferentes elementos que forman una trama, como el bit de inicio, los bits de datos, el bit de paridad y los bits de parada. Además, se comprobó que los datos se transmiten comenzando por el bit menos significativo (LSB).
- Al modificar la paridad en los caracteres, se comprobó que los bits de datos no cambian, sino que únicamente se modifica el bit destinado a la paridad. Al eliminar completamente la paridad, la trama pierde un bit y, por lo tanto, también disminuye el tiempo necesario para transmitir cada carácter. Esto permitió comprobar experimentalmente la relación entre la configuración de la trama y su duración.
- En el análisis de las tramas completas se comprobó que el tiempo total de transmisión depende de la cantidad de bits que contiene cada carácter y de la velocidad de transmisión.
- En general, las actividades realizadas permitieron relacionar los conceptos teóricos de la comunicación UART con mediciones reales realizadas mediante el osciloscopio. El uso de diferentes velocidades, cantidades de bits, tipos de paridad y bits de parada permitió observar cómo cada parámetro modifica la estructura y el tiempo de una transmisión, ayudando a comprender mejor el funcionamiento de la comunicación serial en un microcontrolador.
- El reto permitió implementar y comprobar una comunicación serial entre una Raspberry Pi Pico 2 W, utilizada como transmisor, y una Raspberry Pi Pico, utilizada como receptor. En conjunto con las etapas anteriores, se verificó la interconexión de las líneas de comunicación, la transmisión y recepción de caracteres, la confirmación mediante ACK y finalmente la transferencia de archivos mediante UART.
- La etapa de transmisión de archivos permitió observar experimentalmente la relación entre el intervalo de envío, la cantidad de información y la confiabilidad de la comunicación. Los resultados demostraron que con intervalos de 1 s y 0.1 s se obtuvieron transferencias completas en los casos evaluados, mientras que al utilizar 0.01 s y 0 s aparecieron pérdidas importantes, especialmente al aumentar el número de líneas.
- En conclusión, la práctica permitió comprobar que una comunicación serial no depende únicamente de que el transmisor envíe correctamente los datos, sino también de que el receptor tenga suficiente capacidad para recibir, procesar y almacenar la información. Por ello, seleccionar un intervalo adecuado y establecer un mecanismo de inicio, finalización y confirmación de la transferencia son aspectos fundamentales para obtener una comunicación confiable.

VI. BIBLIOGRAFÍA

- [1] «PUSR,» 18 Noviembre 2024. [En línea]. Available:
] <https://www.pusr.com/blog/Que-es-la-Comunicacio-n-Serial-y-su-Principio-de-Funcionamiento>. [Último acceso: 17 Agosto 2026].
- [2] «Luis Llamas,» [En línea]. Available:
] <https://www.luisllamas.es/como-usar-puerto-serie-uart-raspberry-pi/>. [Último acceso: 17 Agosto 2026].
- [3] «Micropython,» [En línea]. Available:
] <https://docs.micropython.org/en/latest/library/machine.UART.html>. [Último acceso: 2026 Agosto 2026].
- [4] «Raspberry Pi,» [En línea]. Available:
] <https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html>. [Último acceso: 17 Agosto 2026].

VII. ANEXOS

1. Código del punto 3.3:

```
import machine
import utime
from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)
uart = UART(0, baudrate=9600, bits=8, parity=0, tx=Pin(0), rx=Pin(1))

while True:
    led.on()
    uart.write("U")
    utime.sleep(1)
    led.off()
    utime.sleep(1)
```

2. Código del punto 3.4:

Para 1200 baudios:

```
import machine
import utime
from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)
uart = UART(0, baudrate=1200, bits=8, parity=0, tx=Pin(0), rx=Pin(1))

while True:
    led.on()
    uart.write("Y")
    utime.sleep(1)
    led.off()
    utime.sleep(1)
```

Para 4800 baudios:

```
import utime

from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=4800, bits=8, parity=0, tx=Pin(0), rx=Pin(1))

while True:
    led.on()
    uart.write("Y")
    utime.sleep(1)
    led.off()
    utime.sleep(1)
```

Para 9600 baudios:

```
import machine
import utime
from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=9600, bits=8, parity=0, tx=Pin(0), rx=Pin(1))

while True:
    led.on()
    uart.write("Y")
    utime.sleep(1)
    led.off()
    utime.sleep(1)
```

Para 38400 baudios:

```
import machine
import utime
from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=38400, bits=8, parity=0, tx=Pin(0), rx=Pin(1))

while True:
```

```

led.on()

uart.write("Y")

utime.sleep(1)

led.off()

utime.sleep(1)

```

Para 57600 baudios:

```

import machine

import utime

from machine import Pin, UART

```

```

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=57600, bits=8, parity=0, tx=Pin(0),
rx=Pin(1))

```

```

while True:

    led.on()

    uart.write("Y")

    utime.sleep(1)

    led.off()

    utime.sleep(1)

```

Para 115200 baudios:

```

import machine

import utime

from machine import Pin, UART

```

```

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=115200, bits=8, parity=0, tx=Pin(0),
rx=Pin(1))

```

```

while True:

    led.on()

    uart.write("Y")

    utime.sleep(1)

    led.off()

    utime.sleep(1)

```

3. Código para el punto 3.5:

Para la O:

```

import machine

import utime

```

```

from machine import Pin, UART

```

```

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=9600, bits=8, parity=None, stop=1,
tx=Pin(0), rx=Pin(1))

```

```

while True:
    led.on()
    uart.write("O")
    utime.sleep(1)
    led.off()
    utime.sleep(1)

```

Para el @:

```

import machine

import utime

from machine import Pin, UART

```

```

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=4800, bits=8, parity=0, stop=1,
tx=Pin(0), rx=Pin(1))

```

```

while True:
    led.on()
    uart.write("@")
    utime.sleep(1)
    led.off()
    utime.sleep(1)

```

Para la S:

```

import machine

import utime

from machine import Pin, UART

```

```

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=38400, bits=8, parity=1, stop=1,
tx=Pin(0), rx=Pin(1))

```

```

while True:
    led.on()
    uart.write("S")
    utime.sleep(1)
    led.off()
    utime.sleep(1)

```

Para ?:

```

import machine

import utime

from machine import Pin, UART

```

```

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=57600, bits=7, parity=None, stop=1,
tx=Pin(0), rx=Pin(1))

```

```

while True:

```

```
led.on()
uart.write("?")
utime.sleep(1)
led.off()
utime.sleep(1)
```

Para W:

```
import machine
import utime
from machine import Pin, UART
```

```
led = machine.Pin("LED", machine.Pin.OUT)
uart = UART(0, baudrate=115200, bits=7, parity=0, stop=1,
tx=Pin(0), rx=Pin(1))
```

```
while True:
    led.on()
    uart.write("W")
    utime.sleep(1)
    led.off()
    utime.sleep(1)
```

4. Código para el punto 3.6:

b)

Para @:

```
import machine
import utime
from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=4800, bits=8, parity=1, stop=1,
tx=Pin(0), rx=Pin(1))

while True:

    led.on()

    uart.write("@")

    utime.sleep(1)

    led.off()

    utime.sleep(1)
```

Para la S:

```
import machine
import utime
from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)
```

```
uart = UART(0, baudrate=38400, bits=8, parity=0, stop=1,
tx=Pin(0), rx=Pin(1))
```

while True:

```
    led.on()

    uart.write("S")

    utime.sleep(1)

    led.off()

    utime.sleep(1)
```

Para la W:

```
import machine
```

```
import utime
```

```
from machine import Pin, UART
```

```
led = machine.Pin("LED", machine.Pin.OUT)
```

```
uart = UART(0, baudrate=115200, bits=7, parity=1, stop=1,
tx=Pin(0), rx=Pin(1))
```

while True:

```
    led.on()

    uart.write("W")

    utime.sleep(1)

    led.off()

    utime.sleep(1)
```

c)

Para @:

```
import machine
```

```
import utime
```

```
from machine import Pin, UART
```

```
led = machine.Pin("LED", machine.Pin.OUT)
```

```
uart = UART(0, baudrate=4800, bits=8, parity=None, stop=1,
tx=Pin(0), rx=Pin(1))
```

while True:

```
    led.on()

    uart.write("@")

    utime.sleep(1)
```

```
led.off()

utime.sleep(1)
```

Para la S:

```
import machine

import utime

from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=38400, bits=8, parity=None, stop=1,
tx=Pin(0), rx=Pin(1))
```

```
while True:

    led.on()

    uart.write("S")

    utime.sleep(1)

    led.off()

    utime.sleep(1)
```

Para la W:

```
import machine

import utime

from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=115200, bits=7, parity=None, stop=1,
tx=Pin(0), rx=Pin(1))
```

```
while True:

    led.on()

    uart.write("W")

    utime.sleep(1)

    led.off()

    utime.sleep(1)
```

5. Código para el punto 3.7:

a)

```
import machine

import utime

from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=600, bits=8, parity=0, stop=1,
tx=Pin(0), rx=Pin(1))

while True:

    led.on()

    uart.write("ABCDEFGHJKLMNOPQRSTUVWXYZ1234567890A
BCDEFGHIJKLMNOPQRSTUVWX")

    utime.sleep(1)

    led.off()

    utime.sleep(1)
```

b)

```
import machine

import utime

from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=600, bits=8, parity=None, stop=1,
tx=Pin(0), rx=Pin(1))

while True:

    led.on()
```

```
uart.write("ABCDEFGHJKLMNOPQRSTUVWXYZ1234567890A
BCDEFGHIJKLMNOPQRSTUVWX")

    utime.sleep(1)

    led.off()

    utime.sleep(1)
```

c)

```
import machine

import utime

from machine import Pin, UART

led = machine.Pin("LED", machine.Pin.OUT)

uart = UART(0, baudrate=57600, bits=7, parity=0, stop=2,
tx=Pin(0), rx=Pin(1))

while True:

    led.on()

    uart.write("UMNG LIDER EN INGENIERIA EN
TELECOMUNICACIONES")
```

```

utime.sleep(1)
led.off()
utime.sleep(1)

```

6. Código para el punto 4:

b) TX

```

from machine import Pin, UART
import time

uart = UART(0, baudrate=9600, bits=8,
            parity=None,
            stop=1, tx=Pin(0), rx=Pin(1))

led = Pin("LED", Pin.OUT)

print("TRANSMISOR listo")
print("Escriba un caracter:")

while True:
    caracter = input("Caracter: ")

    if len(caracter) > 0:
        uart.write(caracter[0])
        print("Enviado:", caracter[0])

        while not uart.any():
            time.sleep(0.01)

        respuesta = uart.read(1)

        if respuesta == b"K":
            print("ACK recibido")
            led.on()
            time.sleep(1)
            led.off()
            time.sleep(1)

```

RX

```

from machine import Pin, UART, I2C
import time
import ssd1306

uart = UART(0, baudrate=9600, bits=8,
            parity=None,
            stop=1, tx=Pin(0), rx=Pin(1))

led = Pin("LED", Pin.OUT)

i2c = I2C(0, sda=Pin(4), scl=Pin(5), freq=400000)
oled = ssd1306.SSD1306_I2C(128, 64, i2c)

oled.fill(0)
oled.text("RECEPTOR", 25, 0)
oled.text("Esperando...", 10, 20)
oled.show()

print("Receptor listo")

while True:
    if uart.any():
        dato = uart.read(1)

        if dato:
            caracter = dato.decode()
            print("Recibido:", caracter)

```

```

oled.fill(0)
oled.text("RECIBIDO:", 20, 0)
oled.text(caracter, 60, 25)
oled.show()

led.on()
time.sleep(5)
led.off()

uart.write("K")

oled.fill(0)
oled.text("RECEPTOR", 25, 0)
oled.text("Esperando...", 10, 20)
oled.show()

```

c)

CODIGO alfabetos.txt

```

def generar_archivo():
    while True:
        try:
            cantidad = int(input("Numero de lineas (1-1000): "))

            if 1 <= cantidad <= 1000:
                break

            print("El numero debe estar entre 1 y 1000.")

        except:
            print("Ingrese un numero entero valido.")

    with open("alfabetos.txt", "w") as archivo:
        for i in range(1, cantidad + 1):
            linea = "{:04d} ABCDEFGHIJKLMNOPQRSTUVWXYZ\n".format(i)
            archivo.write(linea)

    print("Archivo alfabetos.txt creado.")
    print("Cantidad de lineas:", cantidad)

generar_archivo()

```

CODIGO enviar.py

```

from machine import Pin, UART
import time

uart = UART(
    0,
    baudrate=9600,
    bits=8,
    parity=None,
    stop=1,
    tx=Pin(0),
    rx=Pin(1)
)

print("TRANSMISOR DE ARCHIVO")

with open("alfabetos.txt", "r") as archivo:
    lineas = archivo.readlines()

print("Lineas a enviar:", len(lineas))

while True:
    try:
        intervalo = float(input("Intervalo entre lineas

```

```

(segundos): ")

    if intervalo >= 0:
        break

    print("El intervalo no puede ser negativo.")

except:
    print("Ingrese un numero valido.")

print("Iniciando transmision...")

inicio = time.time()

for i, linea in enumerate(lineas):

    uart.write(linea)

    print("Enviado:", linea.strip())

    if i < len(lineas) - 1:
        time.sleep(intervalo)

uart.write("<<FIN>>\n")
uart.flush()

fin = time.time()

tiempo_total = time.time_diff(fin, inicio) / 1000

print("-----")
print("TRANSMISION TERMINADA")
print("Lineas enviadas:", len(lineas))
print("Intervalo:", intervalo, "s")
print("Tiempo total:", tiempo_total, "s")
print("-----")

```

CODIGO recibir.py

```

from machine import Pin, UART
import time

uart = UART(
    0,
    baudrate=9600,
    bits=8,
    parity=None,
    stop=1,
    tx=Pin(0),
    rx=Pin(1)
)

print("RECEPTOR DE ARCHIVO")
print("Esperando datos...")

archivo = open("recibido.txt", "w")

buffer = ""
contador = 0
inicio = None

while True:

    if uart.any():

        datos = uart.read()

        if datos:

            if inicio is None:
                inicio = time.time()

            buffer += datos.decode()

            while "\n" in buffer:

                posicion = buffer.find("\n")

```

```

        linea = buffer[posicion + 1]
        buffer = buffer[posicion + 1:]

        if linea.strip() == "<<FIN>>":

            archivo.close()

            fin = time.time()
            tiempo_total = time.time_diff(fin,
                inicio) / 1000

            print("-----")

            print("RECEPCION TERMINADA")
            print("Lineas recibidas:",
                contador)

            print("Tiempo total:",
                tiempo_total, "s")

            print("-----")

            while True:
                time.sleep(1)

            archivo.write(linea)
            archivo.flush()

            contador += 1
            print("Recibido:", linea.strip())

```

IMAGENES DE LOS ENVIOS DE CONSOLA

```

Consola x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
TRANSMISOR DE ARCHIVO
Lineas a enviar: 10
Intervalo entre lineas (segundos): 0.1
Iniciando transmision...
Enviado: 0001 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0002 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0003 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0004 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0005 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0006 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0007 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0008 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0009 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0010 ABCDEFGHIJKLMNOPQRSTUVWXYZ
-----
TRANSMISION TERMINADA
Lineas enviadas: 10
Intervalo: 0.1 s
Tiempo total: 0.942 s
>>> |

```

```
Consola x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
TRANSMISOR DE ARCHIVO
Lineas a enviar: 10
Intervalo entre lineas (segundos): 1
Iniciando transmision...
Enviado: 0001 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0002 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0003 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0004 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0005 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0006 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0007 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0008 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0009 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0010 ABCDEFGHIJKLMNOPQRSTUVWXYZ
-----
TRANSMISION TERMINADA
Lineas enviadas: 10
Intervalo: 1.0 s
Tiempo total: 9.042 s
-----

>>> |
```

MicroPython (Raspberry Pi Pico) • Board CDC @ COM7

```
Consola x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
RECEPTOR DE ARCHIVO
Esperando datos...
Recibido: 0001 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0002 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0003 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0004 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0005 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0006 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0007 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0008 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0009 ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

```
Consola x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
TRANSMISOR DE ARCHIVO
Lineas a enviar: 10
Intervalo entre lineas (segundos): 0
Iniciando transmision...
Enviado: 0001 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0002 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0003 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0004 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0005 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0006 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0007 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0008 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0009 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Enviado: 0010 ABCDEFGHIJKLMNOPQRSTUVWXYZ
-----
TRANSMISION TERMINADA
Lineas enviadas: 10
Intervalo: 0.0 s
Tiempo total: 0.304 s
-----

>>>
```

```
Consola x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
RECEPTOR DE ARCHIVO
Esperando datos...
Recibido: 0001 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0002 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0003 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0004 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0005 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0006 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0007 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0008 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0009 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0010 ABCDEFGHIJKLMNOPQRSTUVWXYZ
-----
RECEPCION TERMINADA
Lineas recibidas: 10
Tiempo total: 0.087 s
-----
```

```
Consola x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
RECEPTOR DE ARCHIVO
Esperando datos...
Recibido: 0001 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0002 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0003 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0004 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0005 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0006 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0007 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0008 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0009 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0010 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0011 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0012 ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

```
Consola x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
RECEPTOR DE ARCHIVO
Esperando datos...
Recibido: 0001 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0002 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0003 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0004 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0005 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0006 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0007 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0008 ABCDEFGHIJKLMNOPQRSTUVWXYZ
Recibido: 0009 ABCDEFGHIJKLMNOPQRSTUVWXYZ
```